

Aula 1:

Introdução à Maratona de Programação

Disciplina: Maratona de Programação 1

Profs. Edmilson Marmo e Luiz Olmes

edmarmo@unifei.edu.br, olmes@unifei.edu.br



UNIFEI
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Nas aulas anteriores...

▶ **O QUE JÁ ESTUDAMOS?**

- ▶ Apresentação da Disciplina

▶ **OBJETIVOS:**

- ▶ Introdução à Maratona
- ▶ Os Problemas
- ▶ Formatos de Entrada
- ▶ Shell:
 - ▶ Comandos básicos
 - ▶ Compilação
 - ▶ Correção

O que é a Maratona de Programação?

- ▶ A Maratona de Programação é um evento promovido pela Sociedade Brasileira de Computação desde 1996.
 - ▶ Site: maratona.sbc.org.br (Brasil), icpc.global (mundo)
- ▶ Os times, compostos por exatamente 3 integrantes, representam as instituições de ensino superior. Cada time tem um único computador.
 - ▶ Entrosamento, trabalho em equipe, habilidades distintas.
- ▶ Fases da competição oficial:
 - ▶ Primeira Fase: Regional
 - ▶ Final Brasileira: Brasil, América Central, Caribe, México e América do Sul.
 - ▶ Programadores das Américas: continente Americano.
 - ▶ Final Mundial (ICPC)



Objetivo

Resolver o **maior número** de problemas computacionais...
...no **menor tempo** possível.

Os Problemas

- ▶ Os problemas da competição são normalmente descritos em **quatro** partes:
- ▶ **Enunciado**: varia bastante em complexidade. Pode trazer diretamente a informação do que deve ser feito, ou então contar uma longa história sobre o assunto e desviar a sua atenção. Também pode disfarçar detalhes da solução.
- ▶ **Descrição da entrada**: importantíssima e deve ser respeitada. Contém a descrição dos dados de entrada e a ordem em que estarão dispostos.
- ▶ **Descrição da saída**: contém a formatação exata de como a resposta deve ser impressa. Importantíssima e deve ser respeitada, pois os juízes irão rejeitar respostas corretas, porém mal formatadas.

Os Problemas

- ▶ **Exemplos de entrada e saída:** vêm por último, mostrando valores de entrada e as respostas esperadas para aquelas entradas. Os valores dos exemplos estarão **sempre corretos**.

Os Problemas

- ▶ **Exemplos de entrada e saída:** vêm por último, mostrando valores de entrada e as respostas esperadas para aquelas entradas. Os valores dos exemplos estarão **sempre corretos**.
- ▶ Esta seção constitui uma **pequena amostra** de valores de entrada/saída que serão submetidos ao seu programa para testes.
 - ▶ *E se a minha resposta deu diferente?* 😞
 - ▶ *Seu programa está errado. Corrija-o!* 😎
- ▶ Os conjuntos de testes dos juízes contém valores de entrada/saída adicionais (muitos!), com o objetivo de testar seu programa com valores extremos.
 - ▶ *As baterias de teste dos juízes não estarão disponíveis aos competidores!* 😜

Os Problemas: Exemplo

O problema de soma

Título

Enunciado

São dados dois inteiros a e b . Imprima o valor de $a + b$.

Atenção aos
limites da entrada!

Entrada

A única linha da entrada contém os inteiros a e b ($-100 \leq a, b \leq 100$).

Saída

Imprima o valor de $a + b$.

Atenção ao
formato da saída!

Exemplos:

<i>Ex. de Entrada</i>	<i>Ex. de Saída</i>
7 8	15
-100 100	0
-7 -99	-106

Mas e aí, deu certo?

- ▶ Correção automatizada:

- ▶ Juntamente com o problema, são preparados diversos casos de teste com suas respostas esperadas.

- ▶ Gotta catch'em all:

- ▶ Para que a sua solução seja aceita, é preciso que o seu código gere os mesmos valores presentes na resposta esperada para todos os casos de teste.

- ▶ Time is money:

- ▶ Seu código tem que gerar a resposta dentro de um pequeno intervalo de tempo.
- ▶ Se o limite de tempo for ultrapassado, sua solução não é aceita (ainda que correta!).

Formato de Entrada

- ▶ Há 4 tipos de formatos de entrada principais:

- ▶ Caso de teste único.

- ▶ Ex.: Distância entre dois pontos (Beecrowd 1015)

- ▶ Entrada finalizando em zeros.

- ▶ Ex.: Dama (Beecrowd 1087)

- ▶ Indicação da quantidade de casos.

- ▶ Ex.: Conversa Internacional (Beecrowd 1581)

- ▶ Entrada finalizando com fim de arquivo (EOF).

- ▶ Ex: Soma de Fatoriais (Beecrowd 1161)



Formato de Entrada: Distância entre dois pontos (1015)

- Casos de teste únicos: basta ler cada variável, usando `scanf` ou `cin`.

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
1.0 7.0 5.0 9.0	4.4721

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
-2.5 0.4 12.1 7.3	16.1484

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
2.5 -0.4 -12.2 7.0	16.4575

Formato de Entrada: Distância entre dois pontos (1015)

- Casos de teste únicos: basta ler cada variável, usando `scanf` ou `cin`.

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
1.0 7.0 5.0 9.0	4.4721

```
1. ...
2.
3. float x1, y1, x2, y2;
4.
5. scanf("%f %f %f %f", &x1, &y1, &x2, &y2);
6.
7. // ou, em C++:
8.
9. cin >> x1 >> y1 >> x2 >> y2;
```

Formato de Entrada: Dama (1087)

- ▶ **Finalizando em zeros**: basta ler cada variável usando `scanf` ou `cin`, criar um laço de processamento e, a última instrução do laço é a repetição da leitura.

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
4 4 6 2 3 5 3 5 5 5 4 3 0 0 0 0	1 0 2

Formato de Entrada: Dama (1087)

- ▶ **Finalizando em zeros**: basta ler cada variável usando `scanf` ou `cin`, criar um laço de processamento e, a **última** instrução do laço é a repetição da leitura.

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
4 4 6 2 3 5 3 5 5 5 4 3 0 0 0 0	1 0 2

```
1. int x1, y1, x2, y2;  
2.  
3. scanf("%d %d %d %d", &x1, &y1, &x2, &y2);  
4.  
5. while(x1 || y1 || x2 || y2)  
6. {  
7.     ...  
8.     scanf("%d %d %d %d", &x1, &y1, &x2, &y2);  
9. }
```

Formato de Entrada: Conversa Internacional (1581)

- ▶ **Indicação da quantidade de casos de teste:** basta ler a quantidade **N** de casos de testes e, em um laço que conta **N** vezes, ler as variáveis da entrada.

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
2 3 portugues chines portugues 2 espanhol espanhol	ingles espanhol

Formato de Entrada: Conversa Internacional (1581)

- Indicação da quantidade de casos de teste: basta ler a quantidade **N** de casos de testes e, em um laço que conta **N** vezes, ler as variáveis da entrada.

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
<pre>2 3 portugues chines portugues 2 espanhol espanhol</pre>	<pre>ingles espanhol</pre>

```
1. int n, k;
2.
3. scanf("%d", &n);
4.
5. while(n--)
6. {
7.     scanf("%d", &k);
8.     ...
9. }
```


Formato de Entrada / Saída: Soma de Fatoriais (1161)

- ▶ **Entrada finalizando com fim de arquivo (EOF):** não se sabe quantos casos de teste o problema possui. Neste caso, testa-se o `scanf` ou `cin` em um laço.

<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
4 4 0 0 0 2	48 2 3

Formato de Entrada: Soma de Fatoriais (1161)

- ▶ **Entrada finalizando com fim de arquivo (EOF):** não se sabe quantos casos de teste o problema possui. Neste caso, testa-se o **scanf** ou **cin** em um laço.

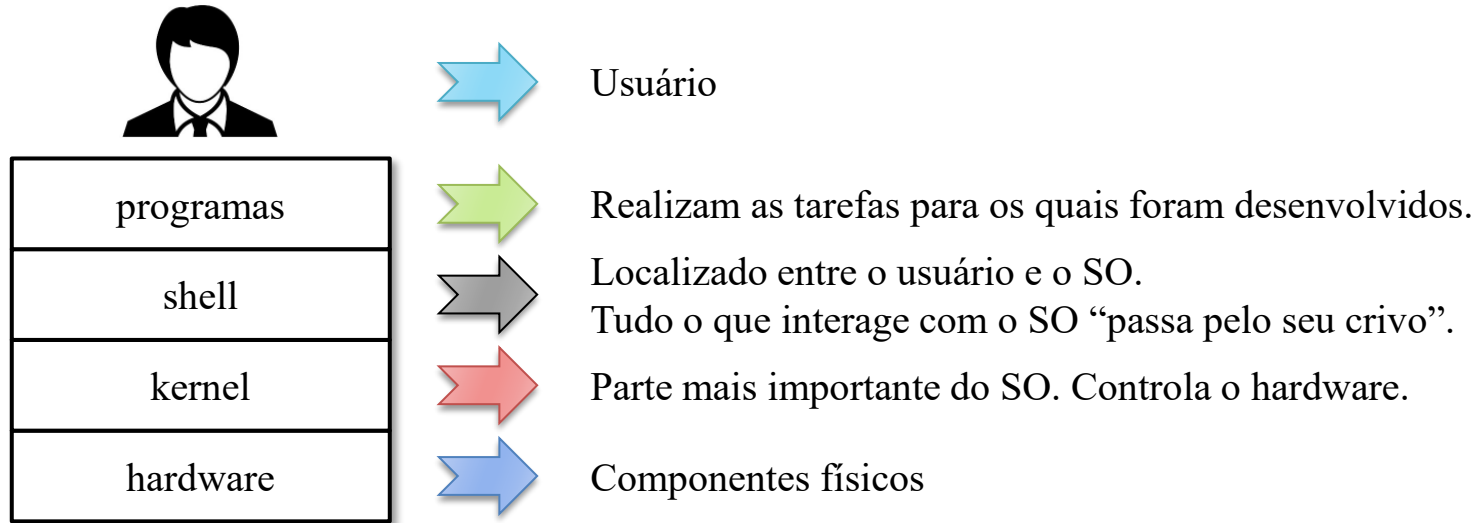
<i>Exemplo de entrada</i>	<i>Exemplo de saída</i>
4 4 0 0 0 2	48 2 3

```
1. long long int a, b;  
2.  
3. while( scanf("%lld %lld", &a, &b) != EOF ) // em C  
4. {  
5.     ...  
6. }  
7.  
8. // ou, em C++:  
9. while( cin >> a >> b ) { ... }
```

Shell

- ▶ O computador disponibilizado aos times para a competição vem com o sistema **Maratona-Linux** instalado.
 - ▶ Ideal apenas para a competição, pois o sistema contém restrições e proteções.
 - ▶ Para o seu uso na disciplina, a sugestão é usar Ubuntu.
- ▶ Trata-se de uma modificação do Ubuntu contendo diversos aplicativos, compiladores para as linguagens aceitas e plugins de segurança previamente configurados.
- ▶ Neste caso, conhecer o básico do Linux e do Shell é fundamental para os competidores.

Shell: o ambiente Linux



Shell: comandos básicos

- ▶ `ls`: lista o conteúdo do diretório (pasta) atual.
- ▶ `pwd`: mostra qual é o diretório atual.
- ▶ `cd`: permite trocar de diretório
 - ▶ Exemplo: `cd /var/www`
 - ▶ `cd ..` sobre um nível na árvore de diretórios.
- ▶ `mv`, `cp`: comandos para mover/copiar arquivos e diretórios.
 - ▶ Sintaxe: `mv origem destino`, `cp origem destino`
- ▶ `rm <nome>`: remove arquivos e diretórios (`rmdir`)

Shell: compilação de programas (C/C++)

- ▶ Embora a correção dos programas submetidos, hoje, seja feita de forma automatizada, por meio de scripts, os juízes (pessoas) irão verificar o resultado do “autojudge” antes de enviar o julgamento ao time.
- ▶ Em caso de dúvidas sobre o resultado, os programas serão compilados e testados manualmente, usando um Shell, da seguinte forma:
- ▶ Compilação (onde o time enviou um arquivo chamado `problemaA.c`):
 - ▶ `gcc -o saida problemaA.c`

Shell: compilação de programas (C/C++)

- ▶ Embora a correção dos programas submetidos, hoje, seja feita de forma automatizada, por meio de scripts, os juízes (pessoas) irão verificar o resultado do “autojudge” antes de enviar o julgamento ao time.
- ▶ Em caso de dúvidas sobre o resultado, os programas serão compilados e testados manualmente, usando um Shell, da seguinte forma:
- ▶ Compilação (onde o time enviou um arquivo chamado problemaA.c):

▶ `gcc -o saida problemaA.c`

compilador C nome do arquivo executável gerado código fonte

Shell: compilação de programas (C/C++)

- ▶ Embora a correção dos programas submetidos, hoje, seja feita de forma automatizada, por meio de scripts, os juízes (pessoas) irão verificar o resultado do “autojudge” antes de enviar o julgamento ao time.
- ▶ Em caso de dúvidas sobre o resultado, os programas serão compilados e testados manualmente, usando um Shell, da seguinte forma:
- ▶ Compilação (onde o time enviou um arquivo chamado problemaA.c):

▶ `gcc -o saida problemaA.c`

compilador C nome do arquivo executável gerado código fonte

Para submissões em C++:
Arquivo fonte terá extensão **.cpp**
Ao invés do **gcc**, invoca-se o **g++**

Shell: correção do programa

- ▶ Após compilado (e não apresentar erros), o programa pode ser executado. Retomando o exemplo anterior:
 - ▶ `gcc -o saida problemaA.c` (compila)
 - ▶ `./saida` (executa)
- ▶ Invocando simplesmente o nome do arquivo executável gerado (`saida`), o programa se abrirá para que os dados sejam digitados e as respostas obtidas.
- ▶ Logicamente, os juízes não irão fazer isso, pois as baterias de testes podem ser imensas (dezenas de milhares de linhas).
- ▶ Os juízes têm à disposição o gabarito (arquivos de texto) de **entrada** e **saída esperada** para os problemas.
 - ▶ Em nosso exemplo, eles se chamarão `problemaA.in`, `problemaA.sol`

Shell: correção do programa

- ▶ Os juízes irão executar o programa da seguinte forma:
 - ▶ `./saida < problemaA.in > problemaA.out`

Shell: correção do programa

- ▶ Os juízes irão executar o programa da seguinte forma:

▶ `./saida` < problemaA.in > problemaA.out,

<: redireciona a entrada
para o arquivo informado

>: redireciona a saída
para o arquivo informado

Shell: correção do programa

- ▶ Os juízes irão executar o programa da seguinte forma:
 - ▶ `./saida < problemaA.in > problemaA.out`
- ▶ E, a seguir, irão verificar a diferença entre os dois arquivos
 - ▶ `problemaA.out`: saída gerada pelo código do time
 - ▶ `problemaA.sol`: gabarito dos juízes com as saídas esperadas
- ▶ da seguinte forma:

Shell: correção do programa

- ▶ Os juízes irão executar o programa da seguinte forma:
 - ▶ `./saida < problemaA.in > problemaA.out`
- ▶ E, a seguir, irão verificar a diferença entre os dois arquivos
 - ▶ `problemaA.out`: saída gerada pelo código do time
 - ▶ `problemaA.sol`: gabarito dos juízes com as saídas esperadas
- ▶ da seguinte forma:
 - ▶ `diff problemaA.out problemaA.sol`
- ▶ Se o resultado do `diff` for nulo (isto é, o comando `diff` não imprimir nada), a solução do time está correta. Senão, a solução apresentou diferenças em relação ao gabarito dos juízes.

Julgamento da solução

- ▶ O código é enviado aos juízes e pode receber as seguintes respostas:
- ▶ *Accepted (YES)*: todos os casos de teste submetidos ao seu programa produziram respostas idênticas ao gabarito dos juízes. Balão! 
- ▶ *Wrong answer*: há um ou mais casos de teste (geralmente vários) que não obtiveram a resposta esperada.
 - ▶ Não são dados detalhes sobre o que está errado.
- ▶ *Time Limit Exceeded*: em um ou mais casos de teste, o programa executou por mais tempo que o máximo permitido.
 - ▶ Este erro não significa que as respostas produzidas estão corretas.

Julgamento da solução

- ▶ *Compilation error*: seu programa está com erro de sintaxe.
 - ▶ *NUNCA FAÇA ISSO!!!!*
 - ▶ Verifique variáveis, protótipos de funções, arquivos de cabeçalho, etc.
- ▶ *Runtime error*: o programa “dá crash” durante a execução.
 - ▶ Causas comuns: divisão por zero, violação de limites de vetor, violação de memória, referências inválidas de ponteiros, uso de mais memória que o permitido, etc.
- ▶ *Presentation error*: seu programa produz a resposta correta, porém ela está mal formatada.
 - ▶ Causas comuns: espaços a esquerda/direita, pula linha a mais/menos, etc. Embora a resposta possa estar correta, a solução não é aceita.

Dúvidas?



Tarefa (para casa): fazer os seguintes problemas no Beecrowd

- ▶ 1015 - Distância entre dois Pontos
- ▶ 1017 - Gasto de Combustível
- ▶ 1021 - Notas e Moedas
- ▶ 1046 - Tempo de Jogo
- ▶ 1087 - Dama
- ▶ 1091 - Divisão da Nlogônia
- ▶ 1140 - Flores Florescem da França
- ▶ 1144 - Sequência Lógica
- ▶ 1161 - Soma de Fatoriais
- ▶ 1163 - Angry Ducks
- ▶ 1318 - Bilhetes Falsos
- ▶ 1367 - Ajude!
- ▶ 1410 - Ele está impedido!
- ▶ 1581 - Conversa Internacional
- ▶ 3106 - Competição de Códigos

Aguarde instruções de entrega!

É esperado que até a próxima aula
você tenha feito a tarefa.

O prazo de entrega será reduzido
quando divulgado.

Aula 1:

Introdução à Maratona de Programação

Disciplina: Maratona de Programação 1

Profs. Edmilson Marmo e Luiz Olmes

edmarmo@unifei.edu.br, olmes@unifei.edu.br



UNIFEI
UNIVERSIDADE FEDERAL DE ITAJUBÁ